AegisOps

Self-healing cloud platform & AI cyber-defense

Contracts, schemas, end-to-end pipeline, 16-week plan and how to start

Source	Dossier de Choix de Projet et Architecture — Idea n°1 (AegisOps), aligned with the official course rules
Team	3 members. Each member owns one Spring Boot service (all core technologies) + one niche engine
Member 1	Spring Boot + **Go** — Telemetry & Chaos Engineering engine
Member 2	Spring Boot + **Quarkus** — Reactive anomaly shield & dynamic firewall
Member 3	Spring Boot + **LangChain4j** — AI Incident Commander (autonomous SRE)
Communication	REST (entry) · gRPC (fast synchronous) · Kafka (asynchronous events)

Contents

- 1. The project in simple words + demo scenario
- 2. Team organigram and responsibilities
- 3. Global architecture
- 4. The end-to-end pipeline (sequence)
- 5. Contracts — repository layout, conventions and ports
- 6. Contracts — gRPC (.proto files)
- 7. Contracts — Kafka events
- 8. Contracts — REST APIs
- 9. Contracts — databases (JPA entities)
- 10. Security
- 11. Resilience, saga and observability
- 12. Testing strategy and CI/CD
- 13. 16-week plan with tasks per member
- 14. How to start (week 0 – week 1, with commands)
- 15. Definition of done, risks and deliverables

1. The project in simple words

Companies run many servers. Servers break (memory leaks, CPU overload) and get attacked (DDoS floods). Normally a human SRE engineer is woken up, looks at dashboards, reads the procedure and repairs. **AegisOps does this automatically.**

- **Member 1 — the sensors and the chaos button (Spring + Go).** Go simulates a fleet of servers and measures them many times per second (CPU, RAM, latency, traffic). It can also inject failures on purpose (Chaos Engineering). Spring stores the server inventory, the network topology and the chaos scenarios.
- **Member 2 — the shield (Spring + Quarkus).** Quarkus reads the stream of measurements in real time and detects attacks or failures in milliseconds. Spring stores the security rules, thresholds, the IP blocklist and the audit history.
- **Member 3 — the AI firefighter (Spring + LangChain4j).** When an alert arrives, an AI agent reads the repair procedures (RAG), decides what to do and calls tools: block the attacker IP (Member 2), isolate or restart the server (Member 1). Then it checks that everything is fine and writes the post-mortem report.

Demo scenario (the target of the whole project)

#	What the jury sees	Who
1	Dashboard: 6 simulated servers, all green. Grafana shows CPU/RAM/latency.	M1
2	Admin clicks *Run scenario "DDoS on web-01"*: 3 attacker IPs flood web-01 with traffic.	M1 Spring → Go
3	In < 1 second Quarkus detects 2,400 req/s from 3 IPs (threshold 500) → CriticalAnomalyDetectedEvent.	M2 Quarkus
4	The AI commander receives the alert, retrieves runbook "RB-01 DDoS mitigation" and decides: block the 3 IPs, then verify.	M3
5	The 3 IPs are blocked; Go stops their traffic; latency goes back to normal; incident RESOLVED.	M3 → M2 → Go
6	Second scenario: memory leak on db-01 → AI isolates the server, restarts it, verifies, reintegrates it.	M3 → M1 → Go
7	Resilience demo: stop Ollama → the circuit breaker opens → rule-based fallback still fixes the incident. Kill one telemetry-service replica → Nginx keeps serving.	All
8	Open the post-mortem written by the AI and the audit trail of every action.	M3, M2

2. Team organigram and responsibilities

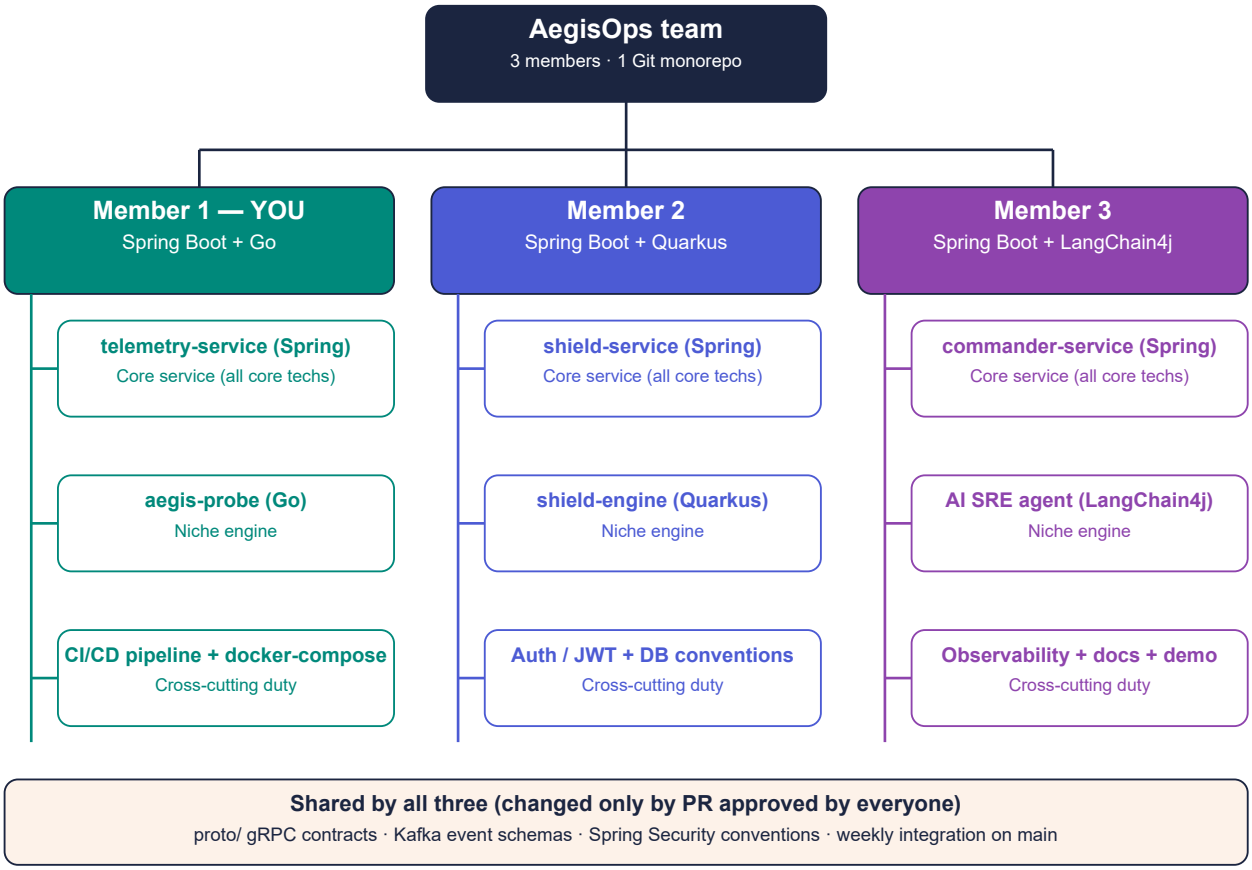


Figure 1 — Team organigram. Every member owns a full vertical slice: a Spring Boot service that covers all core technologies, a niche engine, and one cross-cutting duty.

Requirement	Member 1 (Spring + Go)	Member 2 (Spring + Quarkus)	Member 3 (Spring + LangChain4j)
Spring Boot, MVC, 3-tier, DI	telemetry-service	shield-service	commander-service
Spring Security	JWT resource server + API key for probes	Issues JWT (login), roles	JWT resource server, SRE role for approvals
JPA + PostgreSQL + Flyway	server, topology_link, chaos_scenario, chaos_run	threshold_rule, blocked_ip, anomaly_record, audit_entry, app_user	incident, remediation_step, ai_decision, postmortem, runbook
Servlet / Filter	ApiKeyAuthFilter, TraceIdFilter	ComplianceHeaderFilter, RateLimitFilter	AiRequestInspectionFilter (logs prompts sent to agent)
REST	Servers, topology, chaos API	Rules, blocklist, audit, login	Incidents, post-mortems, chat
gRPC	Client → Go · Server InfraControl	Client → Quarkus · Server Shield	Client → M1 and M2 (AI tools)
Async (Kafka)	Produce chaos.lifecycle · consume remediation.completed	Produce firewall.commands · consume alerts.critical	Consume alerts.critical (@Async) · produce remediation.completed
Logback	JSON + MDC traceId	Audit appender + file rotation	Prompt / latency / response logging
Resilience4j	@CircuitBreaker @Retry on Go calls, cache on inventory	@Retry on Quarkus sync, rate limiter, cache on rules	@TimeLimiter @CircuitBreaker + fallback on LLM, saga
Tests	JUnit 5, Mockito, Testcontainers + go test	JUnit 5, Mockito, @QuarkusTest	JUnit 5, Mockito (LLM mocked), RAG/tool IT
Niche	Go aegis-probe	Quarkus shield-engine	LangChain4j agent

3. Global architecture

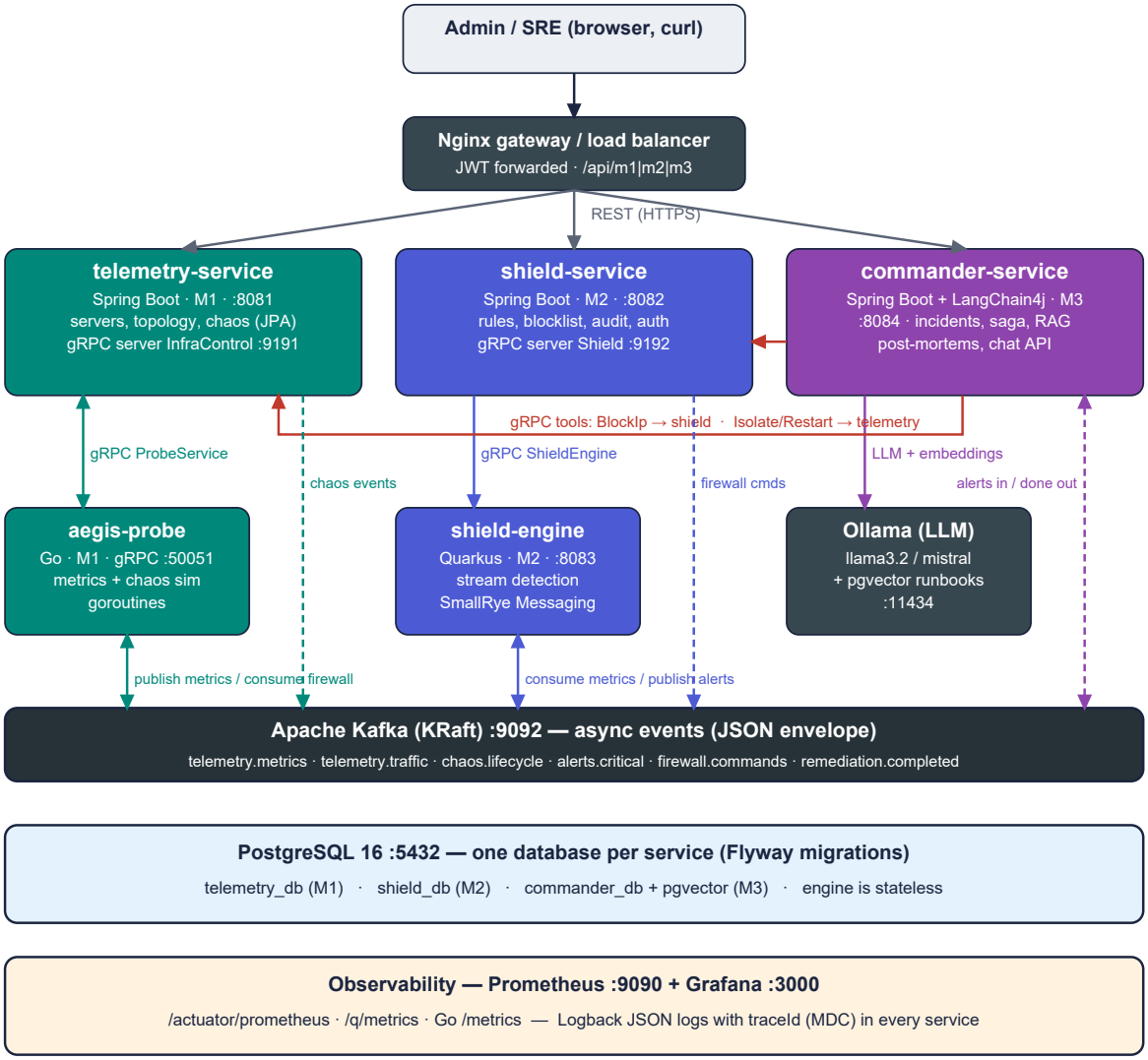


Figure 2 — Component diagram. Solid: synchronous (REST/gRPC). Dashed: Kafka. Red: gRPC calls made by the AI agent tools. Colours = owner (green M1, blue M2, purple M3).

Deployable units

Unit	Owner	Tech	Ports	Replicas (W12)
telemetry-service	M1	Spring Boot 3, Java 21	HTTP 8081 · gRPC 9191	2 (behind Nginx)
aegis-probe	M1	Go 1.23	gRPC 50051 · HTTP 2112 (/metrics, /healthz)	1
shield-service	M2	Spring Boot 3, Java 21	HTTP 8082 · gRPC 9192	2
shield-engine	M2	Quarkus 3 (JVM + native)	HTTP/gRPC 8083	2 (Kafka consumer group)
commander-service	M3	Spring Boot 3 + LangChain4j	HTTP 8084	1–2 (incident lock in DB)
nginx	M1 (CI/infra duty)	Nginx	80	1
kafka · postgres · ollama · prometheus · grafana	shared (M3: monitoring)	apache/kafka, pgvector/pg16, ollama, prom, grafana	9092 · 5432 · 11434 · 9090 · 3000	1 each

4. The end-to-end pipeline

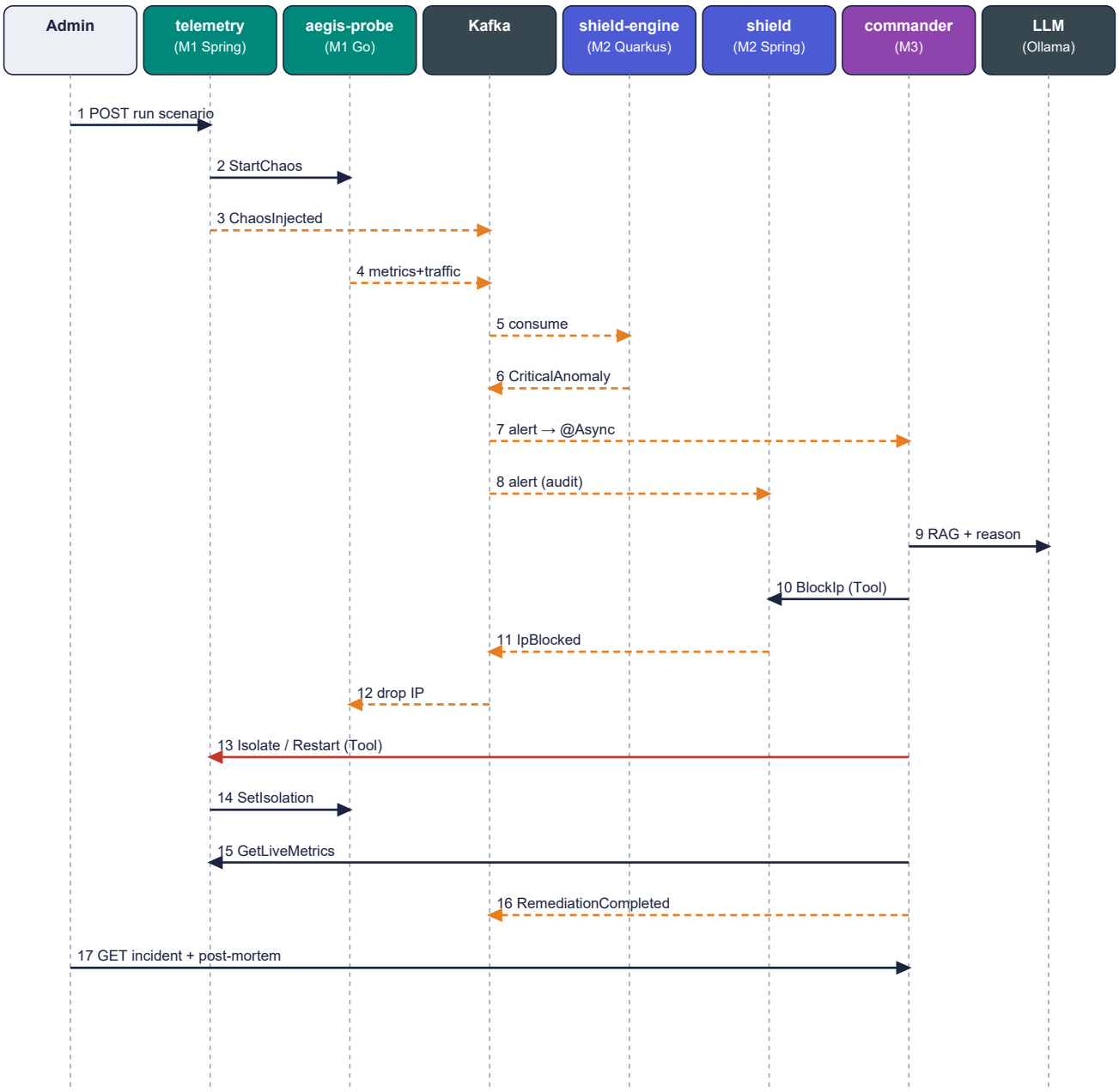


Figure 3 — DDoS scenario, from the chaos button to the post-mortem. Solid = sync (REST/gRPC), dashed orange = Kafka, red = AI tool calls.

#	Step	Contract used
1	Admin runs a chaos scenario.	REST POST /api/v1/chaos/scenarios/{id}/run
2	telemetry-service asks the Go probe to start the chaos.	gRPC ProbeService.StartChaos
3	telemetry-service records the run (JPA) and announces it.	Kafka aegis.chaos.lifecycle — ChaosInjectedEvent
4	Go publishes measurements every 500 ms per server + traffic per source IP every second.	aegis.telemetry.metrics / aegis.telemetry.traffic
5–6	Quarkus evaluates sliding windows against thresholds; when broken, publishes an alert.	aegis.alerts.critical — CriticalAnomalyDetectedEvent
7	commander-service consumes the alert and opens an incident asynchronously (@Async).	Kafka consumer group commander
8	shield-service stores the anomaly in its audit history.	Kafka consumer group shield

#	Step	Contract used
9	The agent retrieves runbooks (RAG) and asks the LLM for a plan (structured output).	LangChain4j AiServices + Ollama
10–12	Tool BlockIp → shield-service persists blocked_ip and publishes a firewall command; Go drops the traffic; Quarkus ignores blocked IPs.	gRPC ShieldService.BlockIp → aegis.firewall.commands
13–14	Tool IsolateServer / RestartServer → telemetry-service → Go.	gRPC InfraControlService → ProbeService
15	Verification: metrics back under thresholds for 3 consecutive checks.	gRPC InfraControlService.GetLiveMetrics
16	Incident closed; everyone is informed.	aegis.remediation.completed — AutoRemediationCompletedEvent
17	Admin reads the incident timeline and the AI post-mortem.	REST GET /api/v1/incidents/{id}

5. Contracts — repository layout, conventions and ports

Repository layout (one Git monorepo)

```
aegisops/
├── proto/aegisops/v1/          # gRPC contracts (shared, PR approved by all 3)
│   ├── common.proto  probe.proto  infra_control.proto  shield.proto  shield_engine.proto
├── events/                     # Kafka event JSON Schemas + examples (shared)
├── telemetry-service/          # M1  Spring Boot  (Maven)
├── aegis-probe/                # M1  Go module    (cmd/, internal/, gen/)
├── shield-service/             # M2  Spring Boot
├── shield-engine/              # M2  Quarkus
├── commander-service/          # M3  Spring Boot + LangChain4j  (src/main/resources/runbooks/*.md)
├── deploy/                     # docker-compose.yml, compose.prod.yml, nginx.conf, prometheus.yml, grafana/
├── docs/                       # architecture.md, diagrams, NicheIntegration-M1/M2/M3.md, weekly-logs/
├── .github/workflows/ci.yml
├── buf.yaml  buf.gen.yaml  pom.xml (parent, modules)  README.md
```

Naming and technical conventions

Topic	Rule
Java packages	`io.aegisops.telemetry`, `io.aegisops.shield`, `io.aegisops.commander`; generated gRPC: `io.aegisops.proto.v1`
Layers (every Spring service)	`api/` controllers + DTOs · `service/` business · `repository/` Spring Data · `domain/` entities · `grpc/` servers/clients · `messaging/` Kafka · `config/`
REST	Prefix `/api/v1`, JSON camelCase, ISO-8601 UTC dates, errors as RFC 7807 `application/problem+json`, pagination `?page=&size=`
IDs	UUID strings everywhere; servers have a readable id (`web-01`, `db-01`)
Headers	`Authorization: Bearer <JWT>` (humans) · `X-API-Key` (probe) · `X-Trace-Id` (all) · gRPC metadata `x-trace-id`, `x-internal-token`
Kafka	Topics `aegis.<domain>.<name>`, key = serverId, JSON envelope (section 7), dead-letter topic `<topic>.dlq`
Commits	Conventional Commits: `feat(probe): add memory leak chaos`, `fix(shield): ...`, scopes = probe, telemetry, shield, engine, commander, proto, deploy
Branches	`main` protected · `feat/<scope>-<topic>` · `fix/...` · PR + 1 review + green CI to merge

Ports and environment variables

Variable	Used by	Example
`SPRING_DATASOURCE_URL / USERNAME / PASSWORD`	3 Spring services	`jdbc:postgresql://postgres:5432/telemetry_db`
`KAFKA_BOOTSTRAP_SERVERS`	all	`kafka:9092`
`AEGIS_JWT_SECRET`	3 Spring services	`(256-bit secret, from Docker secret / Vault)`

Variable	Used by	Example
`AEGIS_INTERNAL_TOKEN`	all gRPC servers and clients	`(shared secret for service-to-service calls)`
`PROBE_GRPC_ADDR`	telemetry-service	`aegis-probe:50051`
`TELEMETRY_GRPC_ADDR / SHIELD_GRPC_ADDR`	commander-service	`telemetry-service:9191 / shield-service:9192`
`ENGINE_GRPC_ADDR`	shield-service	`shield-engine:8083`
`OLLAMA_BASE_URL / OLLAMA_MODEL`	commander-service	`http://ollama:11434 / llama3.2`
`PROBE_SERVERS / PROBE_TICK_MS`	aegis-probe	`web-01,web-02,api-01,db-01,cache-01,worker-01 / 500`

6. Contracts — gRPC (.proto files)

All files live in `proto/aegisops/v1/`, package `aegisops.v1`. Rule: never change or reuse a field number; only add new fields. `buf breaking` runs in CI to enforce it. Every call carries metadata `x-trace-id` and `x-internal-token`.

Service	Server (owner)	Client(s)	Purpose
ProbeService	aegis-probe (M1 Go)	telemetry-service (M1)	Control the simulated fleet and chaos
InfraControlService	telemetry-service (M1)	commander-service (M3)	AI tools on servers: isolate, restart, metrics
ShieldService	shield-service (M2)	commander-service (M3)	AI tools on security: block IP, reputation
ShieldEngineService	shield-engine (M2 Quarkus)	shield-service (M2)	Push rules / blocklist to the engine, stats

common.proto

```
syntax = "proto3";
package aegisops.v1;
option java_package = "io.aegisops.proto.v1";
option java_multiple_files = true;
option go_package = "github.com/aegisops/aegisops/aegis-probe/gen/aegisops/v1;aegisv1";

message ServerRef { string server_id = 1; }
message Empty {}

enum Severity { SEVERITY_UNSPECIFIED = 0; LOW = 1; MEDIUM = 2; HIGH = 3; CRITICAL = 4; }

message MetricSample {
  string server_id        = 1;
  int64  timestamp_ms     = 2;
  double cpu_percent      = 3;   // 0..100
  double mem_percent      = 4;   // 0..100
  double latency_ms       = 5;   // p95 over the last tick
  double error_rate       = 6;   // 0..1
  int32  requests_per_sec = 7;
  string status           = 8;   // UP | DEGRADED | DOWN | ISOLATED
}

message ActionResult {
  bool   success   = 1;
  string message   = 2;
  string action_id = 3;          // UUID, echoed in logs and audit
}
```

probe.proto — server: Go (M1) · client: telemetry-service (M1)

```
import "aegisops/v1/common.proto";

service ProbeService {
  rpc RegisterServer(RegisterServerRequest) returns (ActionResult);
  rpc DeregisterServer(ServerRef)           returns (ActionResult);
  rpc StartChaos(ChaosRequest)              returns (ChaosHandle);
```

```
  rpc StopChaos(ChaosHandle)                returns (ActionResult);
  rpc RestartServer(ServerRef)              returns (ActionResult);
  rpc SetIsolation(IsolationRequest)        returns (ActionResult);
  rpc GetSnapshot(ServerRef)                returns (MetricSample);
  rpc StreamMetrics(StreamMetricsRequest)   returns (stream MetricSample);  // live dashboard
}

enum ChaosType {
  CHAOS_TYPE_UNSPECIFIED = 0; CPU_SPIKE = 1; MEMORY_LEAK = 2;
  LATENCY_INJECTION = 3; DDOS_FLOOD = 4; CRASH = 5;
}
message RegisterServerRequest { string server_id = 1; string name = 2; string zone = 3;
                                double baseline_cpu = 4; double baseline_mem = 5; }
message ChaosRequest {
  string scenario_id = 1; string server_id = 2; ChaosType type = 3;
  double intensity   = 4;            // 0..1
  int32  duration_s  = 5;            // 0 = until StopChaos
  repeated string attacker_ips = 6;  // DDOS_FLOOD only
}
message ChaosHandle          { string chaos_id = 1; }
message IsolationRequest     { string server_id = 1; bool isolated = 2; string reason = 3; }
message StreamMetricsRequest { string server_id = 1; }   // empty = all servers
```

infra_control.proto — server: telemetry-service (M1) · client: commander-service (M3)

```
service InfraControlService {
  rpc ListServers(Empty)                    returns (ServerList);
  rpc GetServer(ServerRef)                  returns (ServerInfo);
  rpc GetLiveMetrics(ServerRef)             returns (MetricSample);
  rpc IsolateServer(RemediationRequest)     returns (ActionResult);
  rpc ReintegrateServer(RemediationRequest) returns (ActionResult);
  rpc RestartServer(RemediationRequest)     returns (ActionResult);
  rpc StopChaos(ServerRef)                  returns (ActionResult);
}
message RemediationRequest {
  string server_id = 1; string incident_id = 2; string reason = 3;
  string idempotency_key = 4;   // incidentId + step order: same key twice = executed once
}
message ServerInfo { string server_id = 1; string name = 2; string zone = 3;
                     string status = 4; repeated string neighbours = 5; }
message ServerList { repeated ServerInfo servers = 1; }
```

shield.proto — server: shield-service (M2) · client: commander-service (M3)

```
service ShieldService {
  rpc BlockIp(BlockIpRequest)         returns (ActionResult);
  rpc UnblockIp(UnblockIpRequest)     returns (ActionResult);
  rpc GetIpReputation(IpRef)          returns (IpReputation);
  rpc ListActiveAnomalies(ServerRef)  returns (AnomalyList);
}
message BlockIpRequest {
  string ip = 1; string server_id = 2; string reason = 3; string incident_id = 4;
  int32  ttl_minutes = 5;            // 0 = permanent
  string idempotency_key = 6;
}
message UnblockIpRequest { string ip = 1; string incident_id = 2; string idempotency_key = 3; }
message IpRef            { string ip = 1; }
message IpReputation     { string ip = 1; bool blocked = 2; int32 anomalies_last_24h = 3; }
message Anomaly          { string anomaly_id = 1; string server_id = 2; string type = 3;
                           Severity severity = 4; int64 detected_at_ms = 5; }
message AnomalyList      { repeated Anomaly anomalies = 1; }
```

shield_engine.proto — server: shield-engine (M2 Quarkus) · client: shield-service (M2)

```
service ShieldEngineService {
  rpc SyncRules(RuleSet)       returns (ActionResult);   // called when rules change in shield-service
  rpc SyncBlocklist(Blocklist) returns (ActionResult);
  rpc GetEngineStats(Empty)    returns (EngineStats);
}
message ThresholdRule {
  string rule_id = 1;
  string metric  = 2;      // cpu_percent | mem_percent | latency_ms | error_rate | rps_per_ip | mem_slope
  string operator = 3;     // GT | LT
  double value   = 4;
```

```
  int32  window_s = 5;     // sliding window length
  Severity severity = 6;
  string anomaly_type = 7; // DDOS | MEMORY_LEAK | CPU_SATURATION | LATENCY_DEGRADATION | SERVER_DOWN
}
message RuleSet     { int64 version = 1; repeated ThresholdRule rules = 2; }
message Blocklist   { int64 version = 1; repeated string ips = 2; }
message EngineStats { int64 events_per_sec = 1; int64 alerts_last_hour = 2; int64 rules_version = 3; }
```

7. Contracts — Kafka events

Events are JSON (easy in Go and Java). Each topic has a JSON Schema in `events/` and an example file used by the contract tests. Every message uses the same envelope; the Kafka key is the `serverId` so that all events of one server stay in order (same partition).

```
{
  "eventId":    "6f1c2e7a-...",            // UUID — consumers ignore an eventId already processed
  "eventType":  "CriticalAnomalyDetectedEvent",
  "version":    1,
  "occurredAt": "2026-11-09T14:03:12.120Z",
  "traceId":    "a1b2c3d4e5f6",            // also sent as Kafka header x-trace-id
  "source":     "shield-engine",
  "payload":    { ... }                    // see below
}
```

Topic	Event(s)	Producer	Consumers	Partitions
`aegis.telemetry.metrics`	TelemetryMetricEvent	aegis-probe (Go)	shield-engine	6
`aegis.telemetry.traffic`	TrafficSampleEvent	aegis-probe (Go)	shield-engine	6
`aegis.chaos.lifecycle`	ChaosInjectedEvent, ChaosStoppedEvent	telemetry-service	commander, shield-service	3
`aegis.alerts.critical`	CriticalAnomalyDetectedEvent	shield-engine	commander, shield-service	3
`aegis.firewall.commands`	IpBlockedEvent, IpUnblockedEvent	shield-service	aegis-probe, shield-engine	3
`aegis.remediation.completed`	AutoRemediationCompletedEvent	commander-service	telemetry-service, shield-service	3

Payloads

```
// TelemetryMetricEvent  (every 500 ms per server)
{ "serverId": "web-01", "cpuPercent": 37.5, "memPercent": 61.2, "latencyMs": 84.0,
  "errorRate": 0.004, "requestsPerSec": 312, "status": "UP" }

// TrafficSampleEvent  (every 1 s per server)
{ "serverId": "web-01", "windowStart": "2026-11-09T14:03:11Z", "windowSeconds": 1,
  "bySourceIp": [ { "ip": "203.0.113.7", "requests": 812 }, { "ip": "10.0.0.15", "requests": 4 } ] }

// ChaosInjectedEvent / ChaosStoppedEvent
{ "chaosId": "c-91", "scenarioId": "sc-ddos-web01", "serverId": "web-01", "type": "DDOS_FLOOD",
  "intensity": 0.8, "attackerIps": ["203.0.113.7","203.0.113.8","203.0.113.9"], "startedBy": "admin" }

// CriticalAnomalyDetectedEvent
{ "anomalyId": "an-5521", "serverId": "web-01", "anomalyType": "DDOS", "severity": "CRITICAL",
  "detectedAt": "2026-11-09T14:03:12.050Z", "ruleId": "r-rps-ip", "metric": "rps_per_ip",
  "observedValue": 812, "threshold": 500, "windowSeconds": 5,
  "suspectIps": ["203.0.113.7","203.0.113.8","203.0.113.9"] }

// IpBlockedEvent / IpUnblockedEvent
{ "ip": "203.0.113.7", "serverId": "web-01", "reason": "DDoS source (incident inc-77)",
  "incidentId": "inc-77", "blockedBy": "AI_COMMANDER", "expiresAt": "2026-11-09T15:03:12Z" }

// AutoRemediationCompletedEvent
{ "incidentId": "inc-77", "anomalyId": "an-5521", "serverId": "web-01",
  "status": "RESOLVED",                    // RESOLVED | FAILED | COMPENSATED | ESCALATED
  "rootCause": "Volumetric DDoS from 3 IPs", "runbookId": "RB-01", "decidedBy": "LLM",  // or FALLBACK
  "actions": [ { "order": 1, "type": "BLOCK_IP", "target": "203.0.113.7", "result": "OK",
```

```
                "at": "2026-11-09T14:03:14Z" } ],
  "durationMs": 4120 }
```

Delivery rules

- Producers: `acks=all`, idempotent producer, retries with backoff (Go: kafka-go writer with retries).
- Consumers: at-least-once + idempotency (table `processed_event(event_id)` in each Spring service).
- A message that fails 3 times goes to `<topic>.dlq` with the error in a header (Spring Kafka DefaultErrorHandler).
- Adding a field = compatible. Removing / renaming a field = new `version` + PR approved by all three.

8. Contracts — REST APIs

All behind Nginx: `/api/m1/**` → telemetry, `/api/m2/**` → shield, `/api/m3/**` → commander. Roles: ADMIN, SRE, VIEWER. OpenAPI generated by springdoc at `/swagger-ui.html`.

telemetry-service (M1)

Method	Path	Role	Description
GET	`/api/v1/servers`	VIEWER	List servers (cached 10 s) with current status
POST	`/api/v1/servers`	ADMIN	Register a server (also RegisterServer on Go)
GET / PUT / DELETE	`/api/v1/servers/{id}`	VIEWER / ADMIN	Read, update, remove
GET	`/api/v1/servers/{id}/metrics/live`	VIEWER	Last MetricSample (gRPC GetSnapshot)
GET / POST	`/api/v1/topology/links`	VIEWER / ADMIN	Network topology (which server talks to which)
GET / POST	`/api/v1/chaos/scenarios`	VIEWER / ADMIN	Chaos scenario catalogue
POST	`/api/v1/chaos/scenarios/{id}/run`	ADMIN	Start a scenario → 202 + chaosRunId
POST	`/api/v1/chaos/runs/{runId}/stop`	ADMIN	Stop a running chaos
GET	`/api/v1/chaos/runs`	VIEWER	History of runs
POST	`/api/v1/probe/heartbeat`	API key	Probe heartbeat (protected by ApiKeyAuthFilter)

shield-service (M2)

Method	Path	Role	Description
POST	`/api/v1/auth/login`	public	Username + password → JWT (roles inside)
GET / POST / PUT / DELETE	`/api/v1/rules[/{id}]`	VIEWER / ADMIN	Threshold rules; change → SyncRules to Quarkus
GET / POST	`/api/v1/blocklist`	VIEWER / ADMIN	Blocked IPs (manual block)
DELETE	`/api/v1/blocklist/{ip}`	ADMIN	Unblock
GET	`/api/v1/anomalies?serverId=&from=`	VIEWER	Anomaly history
GET	`/api/v1/audit?actor=&action=`	ADMIN	Regulatory audit trail
GET	`/api/v1/engine/stats`	VIEWER	Quarkus engine stats (gRPC GetEngineStats)

commander-service (M3)

Method	Path	Role	Description
GET	`/api/v1/incidents?status=`	VIEWER	Incident list

Method	Path	Role	Description
GET	`/api/v1/incidents/{id}`	VIEWER	Incident + steps + AI decision
GET	`/api/v1/incidents/{id}/timeline`	VIEWER	Ordered events of the incident
POST	`/api/v1/incidents/{id}/approve | /reject`	SRE	Human decision when AWAITING_APPROVAL
GET	`/api/v1/postmortems/{incidentId}`	VIEWER	Markdown post-mortem written by the AI
GET / POST	`/api/v1/runbooks`	VIEWER / ADMIN	Runbooks (re-indexed into the vector store)
POST	`/api/v1/assistant/chat`	SRE	Conversational API: "why is web-01 slow?" (agent with tools, read-only)

Error format (all services)

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json
{ "type": "https://aegisops.io/errors/chaos-already-running", "title": "Chaos already running",
  "status": 409, "detail": "Server web-01 already has chaos c-91", "traceId": "a1b2c3d4e5f6" }
```

9. Contracts — databases (JPA entities)

One PostgreSQL database per service: no service reads another service's tables — it must use the API or the events. Every table has `id UUID PK`, `created_at`, `updated_at` and `version` (optimistic locking with @Version). Schema changes only through Flyway scripts `V1__init.sql, V2__....`

DB	Table	Main columns	Relations
telemetry_db (M1)	server	server_key (web-01, unique), name, zone, status, baseline_cpu, baseline_mem	1-N chaos_run
	topology_link	from_server_id, to_server_id, link_type (HTTP, DB, CACHE)	N-1 server ×2
	chaos_scenario	name, type, intensity, duration_s, target_server_id, attacker_ips (text[])	N-1 server
	chaos_run	scenario_id, chaos_id (Go handle), status, started_by, started_at, stopped_at	N-1 scenario
	server_status_history	server_id, old_status, new_status, cause, incident_id	N-1 server
shield_db (M2)	app_user / user_role	username, password_hash (BCrypt), enabled / role	N-N
	threshold_rule	metric, operator, value, window_s, severity, anomaly_type, enabled	—
	blocked_ip	ip (unique), reason, source (MANUAL|AI), incident_id, expires_at	—
	anomaly_record	anomaly_id, server_key, type, severity, observed_value, threshold, detected_at	—
	audit_entry	actor, action, target, details (jsonb), trace_id, at	append-only
commander_db (M3)	incident	anomaly_id (unique), server_key, type, severity, status, opened_at, resolved_at	1-N step, 1-1 decision
	remediation_step	incident_id, step_order, action, target, status, compensation_action, attempts, idempotency_key	N-1 incident
	ai_decision	incident_id, model, runbook_id, root_cause, confidence, latency_ms, fallback_used, raw_json	1-1 incident
	postmortem	incident_id, markdown, generated_at	1-1 incident
	runbook / runbook_chunk	title, content / embedding vector(768) (pgvector)	1-N
each Spring DB	processed_event	event_id (PK), topic, processed_at	Kafka idempotency

10. Security

Who / what	Mechanism
Humans (ADMIN, SRE, VIEWER)	Login on shield-service → JWT (HS256, 1 h). The 3 Spring services are OAuth2 resource servers validating the same key; `@PreAuthorize` on sensitive methods (run chaos = ADMIN, approve = SRE).
Go probe → telemetry-service	API key in `X-API-Key` checked by `ApiKeyAuthFilter` (OncePerRequestFilter); keys stored hashed.
Service ↔ service (gRPC)	Internal Docker network + `x-internal-token` checked by a gRPC ServerInterceptor; TLS optional (stretch).
Abuse	`RateLimitFilter` in shield-service (Resilience4j RateLimiter / Bucket4j): 20 req/s per user, 429 otherwise.
AI safety	The LLM never executes anything itself: it returns a plan; the saga only executes actions from a whitelist (BLOCK_IP, ISOLATE, RESTART, REINTEGRATE, STOP_CHAOS) with validated parameters (IP must be in suspectIps, server must exist). Prompt-injection text in logs is never treated as an instruction.
Secrets	Never in Git: `.env` ignored, Docker secrets in compose; Vault optional (course suggestion).

11. Resilience, saga and observability

Pattern	Where exactly	Settings (starting values)
Retry	Spring → Go, Spring → Quarkus, M3 tools → M1/M2, Go → Kafka	3 attempts, exponential backoff 200 ms × 2, only on UNAVAILABLE / DEADLINE_EXCEEDED
Circuit breaker	Same calls + LLM calls (M3)	window 10 calls, 50 % failures → open 15 s, half-open 3 calls
Timeout	Every gRPC call (deadline), LLM call	gRPC 2 s, LLM 20 s (TimeLimiter)
Fallback	LLM down → rule-based runbook mapping; Quarkus down → shield-service keeps rules in DB and retries sync	anomalyType → default runbook
Cache	M1 server inventory, M2 rules + blocklist, M3 runbook retrieval	Caffeine, TTL 10–60 s, evicted on change
Rate limiting	shield-service RateLimitFilter; Go limits chaos intensity	20 req/s per user
Load balancing / HA	Nginx round-robin over 2 replicas of telemetry and shield; Quarkus scaled through Kafka consumer group	health checks /actuator/health
Saga	commander-service orchestrates remediation; each step has a compensation	state persisted in remediation_step
Idempotency	idempotency_key on gRPC actions; processed_event for Kafka	—

Remediation saga (state machine of an incident)

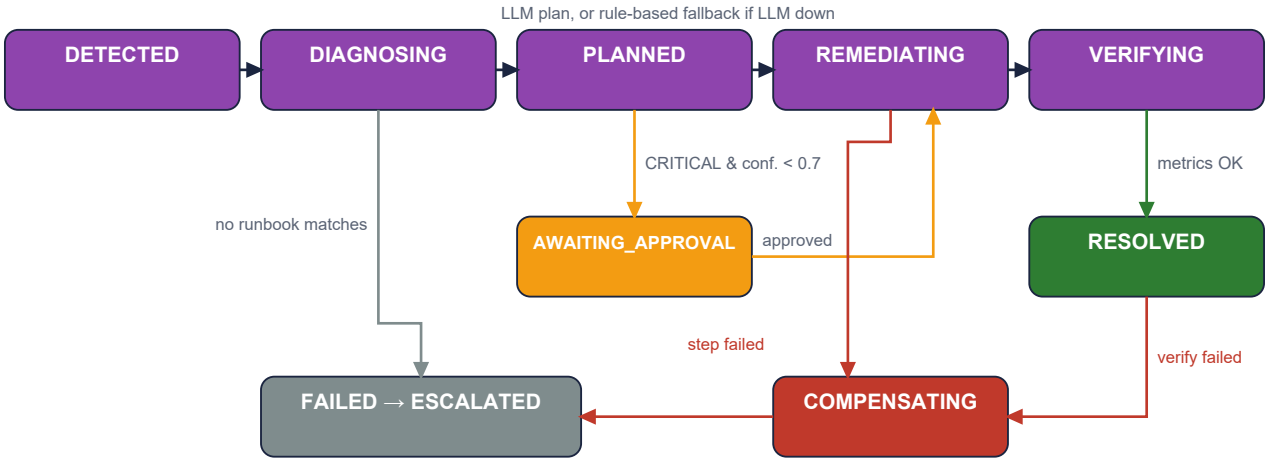


Figure 4 — Incident states in commander-service. On a failed step, the completed steps are compensated in reverse order (UnblockIp, ReintegrateServer).

Runbook	Steps (action → compensation)
RB-01 DDoS mitigation	1. BLOCK_IP each suspect → UNBLOCK_IP · 2. STOP_CHAOS (demo) → — · 3. VERIFY rps < threshold ×3
RB-02 Memory leak	1. ISOLATE server → REINTEGRATE · 2. RESTART server → — · 3. VERIFY mem < 70 % ×3 · 4. REINTEGRATE
RB-03 CPU saturation	1. ISOLATE → REINTEGRATE · 2. STOP_CHAOS · 3. VERIFY · 4. REINTEGRATE
RB-04 Server down	1. RESTART · 2. VERIFY status UP · else ESCALATE

Observability

- **Logback** (`logback-spring.xml`): JSON encoder (logstash-logback-encoder), MDC keys `traceId, incidentId, serverId, user`; M2 adds an audit appender with daily rotation; M3 logs prompt size, model, latency and response summary.
- **Trace id**: created by `TraceIdFilter` (or reused from `X-Trace-Id`), propagated in gRPC metadata, Kafka headers and the event envelope. Go logs with slog using the same key.
- **Metrics**: Micrometer (Spring), Quarkus Micrometer, Go prometheus client → Prometheus → Grafana dashboards: fleet health, events/s, alerts, MTTR, LLM latency, circuit-breaker state.

12. Testing strategy and CI/CD

Level	Member 1	Member 2	Member 3
Unit (many)	Go: chaos generators, metric maths, firewall drop logic (table-driven, -race). Spring: services with Mockito (gRPC client mocked).	Quarkus: window/threshold logic, DDoS per-IP counting. Spring: rule validation, blocklist expiry, RateLimitFilter.	Saga transitions + compensation order, plan validation (whitelist), fallback mapping; LLM mocked with Mockito.
Integration (some)	@SpringBootTest + Testcontainers Postgres/Kafka; in-process gRPC server for Go fake; Go bufconn tests.	@QuarkusTest with Kafka Dev Services; Spring IT with Testcontainers; JWT + roles with MockMvc.	RAG IT with in-memory embedding store; tool calls against WireMock/in-process gRPC fakes of M1/M2.
Contract	proto: buf lint + buf breaking. Events: each consumer validates the example JSON in events/ against its DTO.	same	same

Level	Member 1	Member 2	Member 3
End-to-end (few)	Script: compose up → run DDoS scenario → expect incident RESOLVED in < 30 s; memory-leak scenario; LLM stopped → fallback still resolves.		

Target: ≥ 70 % line coverage on service/ and domain logic (JaCoCo, go test -cover).

CI/CD pipeline (GitHub Actions)

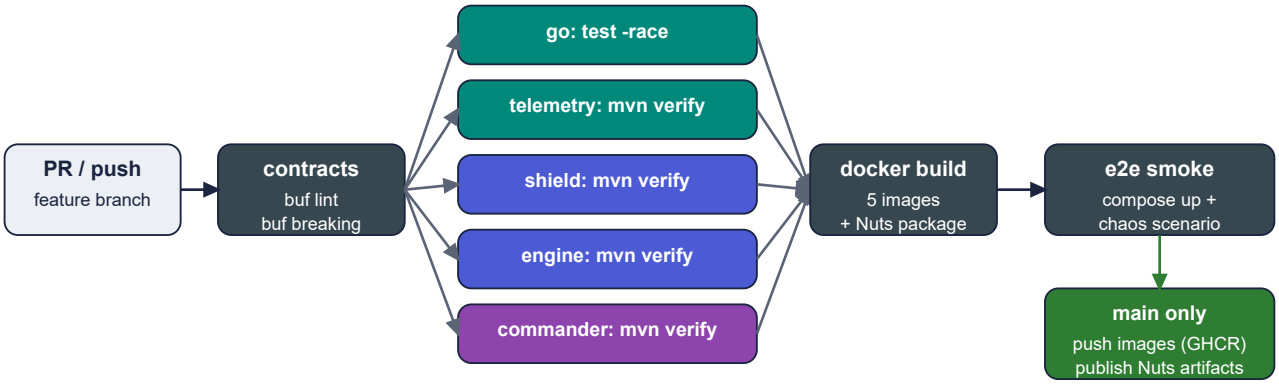


Figure 5 — Pipeline: contracts first, then the five builds in parallel, images, end-to-end smoke test; publishing only on main.

```
# .github/workflows/ci.yml (skeleton)
on:
  pull_request:
  push: { branches: [main] }
jobs:
  contracts:
    runs-on: ubuntu-latest
    steps: [checkout, setup-buf, "buf lint", "buf breaking --against '.git#branch=main'"]
  go:
    needs: contracts
    steps: [checkout, setup-go 1.23, "cd aegis-probe && go vet ./... && go test -race -cover ./..."]
  java:
    needs: contracts
    strategy:
      matrix: { module: [telemetry-service, shield-service, shield-engine, commander-service] }
    steps: [checkout, setup-java 21, "mvn -pl ${{ matrix.module }} -am verify"]
  images: { needs: [go, java], steps: ["docker compose -f deploy/docker-compose.yml build"] }
  e2e:    { needs: images, steps: ["docker compose up -d", "./deploy/e2e.sh", "docker compose down"] }
```

Packaging with Nuts (NAF)

Each Spring service is built as an executable jar and described as a Nuts artifact (id `io.aegisops:telemetry-service#1.0.0`), published to the team Nuts repository by the main pipeline and installed/run with `nuts install` / `nuts run`. **Ask the teacher in week 0 exactly what is expected for NAF/Nuts** (packaging only, or also the Nuts Application Framework APIs) and adapt this section.

13. 16-week plan with tasks per member



Figure 6 — Gantt chart. Red dashed lines = official milestones (W4, W8, W12, W16).

Phase 0 — Kick-off and contracts (W0–W1)

Who	Tasks
All	Read this document together; create the repo; write the 5 .proto files + events/ JSON Schemas; agree on ports, env vars, JWT secret format; create the GitHub Project board; first weekly log.
M1 (you)	Monorepo skeleton, parent pom, buf config, docker-compose (Postgres + Kafka + Ollama), CI skeleton (contracts + build).
M2	Postgres init script (3 DBs + users), JWT format and Spring Security config shared as a snippet.
M3	Install Ollama + model, write runbooks RB-01..RB-04, docs/architecture.md with the diagrams.

Phase 1 — Core backend, DB, REST (W1–W4) → Milestone W4

Who	Tasks
M1	telemetry-service: entities server, topology_link, chaos_scenario, chaos_run + Flyway + REST CRUD + Security + ApiKeyAuthFilter + TraceIdFilter. Go: fleet simulator (goroutine per server), ProbeService gRPC server (GetSnapshot, RegisterServer).
M2	shield-service: auth/login JWT, users, threshold_rule, blocked_ip, audit_entry + REST + RateLimitFilter. Quarkus: project + Kafka consumer printing metrics.
M3	commander-service: incident, remediation_step, postmortem entities + REST + Security; first LangChain4j AiService answering a question about a runbook.
W4 demo	3 Spring services up with DB + secured REST; Go probe answers GetSnapshot via telemetry-service.

Phase 2 — gRPC + Kafka wiring and niche engines (W3–W8) → Milestone W8

Who	Tasks
M1	Go publishes TelemetryMetricEvent + TrafficSampleEvent to Kafka; chaos generators (CPU, memory leak, latency, DDoS, crash); StartChaos/StopChaos/Restart/SetIsolation; InfraControlService gRPC server in Spring; ChaosInjectedEvent; Resilience4j on Go calls.
M2	Quarkus sliding windows + rules → CriticalAnomalyDetectedEvent; ShieldEngineService (SyncRules, SyncBlocklist); ShieldService gRPC server (BlockIp...) + IpBlockedEvent; Go/Quarkus honour the blocklist.
M3	Kafka consumer @Async → incident; RAG (embeddings + pgvector or in-memory); @Tool methods calling M1/M2 via gRPC; structured plan output; Resilience4j + fallback around LLM.
All	Integration tests with Testcontainers; contract tests; first full DDoS flow working.
W8 demo	DDoS scenario → alert → AI blocks IPs → traffic stops.

Phase 3 — Saga, resilience, scalable deployment, monitoring (W8–W12) → Milestone W12

Who	Tasks
M1	compose.prod.yml with 2 replicas + Nginx LB; Dockerfiles (multi-stage, distroless for Go); CI images + e2e job; cache on inventory.
M2	Audit trail complete, blocklist expiry job, rules cache, Quarkus native build + JVM vs native measurements.
M3	Saga orchestrator with compensation + idempotency; AWAITING_APPROVAL flow; post-mortem generation; Prometheus + Grafana dashboards.
W12 demo	Both scenarios + kill a replica + stop the LLM, all visible in Grafana.

Phase 4 — Hardening, documentation, defense (W12–W16) → Milestone W16

Who	Tasks
All	Chaos test suite in CI, bug fixing, coverage ≥ 70 %, README + install guide, NicheIntegration.md (one each), individual report 5–8 pages, weekly logs complete, backup demo video (W15), 2 full rehearsals of the 15-min talk + Q&A.

14. How to start (week 0 – week 1)

Step 1 — Install the tools (each member)

Tool	Version	Check
JDK (Temurin)	21	`java -version`
Maven	3.9+	`mvn -v`
Go (M1 required, others optional)	1.23+	`go version`
Docker Desktop (+ Compose v2)	latest	`docker compose version`
buf + protoc-gen-go + protoc-gen-go-grpc	latest	`buf --version`
Quarkus CLI (M2)	3.x	`quarkus --version`
Ollama (M3, or run in Docker)	latest	`ollama run llama3.2`
Git + GitHub account, IntelliJ / VS Code		`git --version`

Step 2 — Create the repository (M1, day 1)

```
mkdir aegisops && cd aegisops && git init -b main
mkdir -p proto/aegisops/v1 events deploy docs/weekly-logs .github/workflows
# Spring services (Spring Initializr)
for s in telemetry-service shield-service commander-service; do
  curl -s https://start.spring.io/starter.zip -d type=maven-project -d javaVersion=21 \
    -d groupId=io.aegisops -d artifactId=$s -d name=$s \
    -d dependencies=web,security,oauth2-resource-server,data-jpa,postgresql,flyway \
    -d dependencies=validation,actuator,kafka,cache \
    -o $s.zip && unzip -q $s.zip -d $s && rm $s.zip
done
# Quarkus engine (M2)
quarkus create app io.aegisops:shield-engine \
  --extension=grpc,messaging-kafka,smallrye-fault-tolerance,smallrye-health \
  --extension=micrometer-registry-prometheus,rest-jackson
# Go probe (M1)
mkdir aegis-probe && cd aegis-probe && go mod init github.com/aegisops/aegisops/aegis-probe
go get google.golang.org/grpc github.com/segmentio/kafka-go github.com/prometheus/client_golang && cd ..
git add . && git commit -m "chore: bootstrap monorepo skeleton"
```

Extra dependencies to add: Resilience4j (`resilience4j-spring-boot3`), gRPC for Spring (`grpc-server-spring-boot-starter` / `grpc-client-spring-boot-starter` or Spring gRPC), `logstash-logback-encoder`, `springdoc-openapi`, Testcontainers; M3 adds `langchain4j-spring-boot-starter` + `langchain4j-ollama-spring-boot-starter`. Java stubs are generated from `../proto` with a protobuf Maven plugin.

Step 3 — Start the infrastructure (deploy/docker-compose.yml, first version)

```
services:
  postgres:
    image: pgvector/pgvector:pg16
    environment: { POSTGRES_USER: aegis, POSTGRES_PASSWORD: aegis }
    ports: ["5432:5432"]
    volumes: ["./initdb:/docker-entrypoint-initdb.d"]
    # initdb/01-dbs.sql creates the 3 databases: telemetry_db, shield_db, commander_db
  kafka:
    image: apache/kafka:3.8.0          # KRaft mode, no ZooKeeper
    ports: ["9092:9092"]
  ollama:
    image: ollama/ollama
    ports: ["11434:11434"]
    volumes: ["ollama:/root/.ollama"]
volumes: { ollama: {} }

docker compose -f deploy/docker-compose.yml up -d
docker exec -it deploy-ollama-1 ollama pull llama3.2      # M3
buf lint && buf generate                                  # after the .proto files are written
```

Step 4 — Week-1 checklist

Day	Done when…	Who
1	Repo on GitHub, main protected, PR template, board with W1–W4 issues	M1
1–2	5 .proto files written, reviewed in a 1-hour meeting, merged; buf generate works for Go and Maven	All
2	events/ JSON Schemas + one example per event merged	All
3	docker compose up starts Postgres (3 DBs), Kafka, Ollama	M1 + M2
3–4	Each Spring service starts, connects to its DB, runs V1__init.sql, exposes /actuator/health	Each
4	CI green: buf lint + mvn verify + go test on every PR	M1
5	Go probe compiles with generated stubs and answers GetSnapshot; M2 and M3 have their first entity + endpoint	Each
5	Weekly log #1 written by each member; next week's issues assigned	All

Golden rule

Contracts first, code second. As long as the .proto files and the event examples are respected, each member can work alone with fakes of the other services (an in-process gRPC fake, a JSON file published on Kafka). Integrate on main every week — never wait for the end.

15. Definition of done, risks and deliverables

Definition of done (every pull request)

- Code follows the layers (controller → service → repository) and constructor injection.
- Unit tests for the new logic; integration test if it touches DB, Kafka or gRPC; CI green.
- Logs with traceId; errors mapped to problem+json; endpoints secured with the right role.
- Flyway script added for any schema change; contracts unchanged or approved by all three.
- README / docs updated if behaviour or configuration changed; one teammate reviewed.

Risks and mitigations

Risk	Mitigation
The LLM is slow, wrong or unavailable	Small local model, structured output + whitelist validation, timeout + circuit breaker, rule-based fallback, evaluation set of 10 incidents ("correct plan in X/10").
Integration between Go, Quarkus and Spring breaks	Contracts in week 1, buf breaking + event contract tests in CI, weekly integration.

Risk	Mitigation
Too much scope	MVP = DDoS + memory leak scenarios end-to-end. Stretch: real Docker containers instead of simulation, TLS, chat UI, Kubernetes.
A member is blocked	Fakes of every service; pair-programming session each week.
Live demo fails	Seed data, scripted scenarios, backup video, reset script `deploy/reset.sh`.
Unclear requirement (NAF/Nuts, Go accepted as niche)	Ask the teacher in week 0 and record the answer in docs/decisions.md.

Deliverables checklist

Deliverable	Owner	Due
Git repository with meaningful commits from all 3 members	All	continuous
README + installation guide	M3 (docs duty), all contribute	W16
Architecture diagrams (this document's figures, kept up to date in docs/)	M3	W4, updated W12
Deployment scripts / manifests (compose, nginx, prometheus)	M1	W12
NicheIntegration.md (Go / Quarkus / LangChain4j)	each member	W8 draft, W16 final
Individual report 5–8 pages	each member	W16
Weekly log (1 page per week)	each member	every Friday
Live demo + 15-min presentation + 10-min Q&A	All	W16